Espen Oland, 2025
Tom Stian Andersen, 2026

Assignment 1: 3D Visualization of Satellites in Python

1 Introduction

In this assignment the student will learn how to apply Python to visualize satellites orbiting the Earth together with different reference frames and rotations between them.

After completing this assignment the student will learn:

- How to use Python to simulate satellites

2 Preliminaries

Before describing the steps needed to complete todays assignment, some fundamentals are needed. Specifically, the fundamental mathematics needed to simulate a satellite flying in a circular orbit, and how the Earth is rotating around its $\mathbf{z}^i$ axis. For a quick introduction to programming with Python consult Appendix A.

2.1 Reference Frames

In today's assignment there are three reference frames that will be used, namely

- Body frame, has it's origin at the satellite's center of gravity and is fixed to the satellite and is denoted by superscript b.
- Earth Centered Inertial (ECI) frame with origin at the center of Earth, $\mathbf{x}^i$ points along the Vernal Equinox, $\mathbf{z}^i$ along Earth's spin axis, and $\mathbf{y}^i$ completes the right handed system. The ECI frame is denoted by superscript i and is considered an inertial reference frame.
- Earth Centered Earth Fixed (ECEF) frame with origin also at Earth's center and is assumed to be initially aligned with the ECI frame. However, it rotates together with the Earth. The ECEF frame is denoted by superscript e.

2.2 Orbital Mechanics of a Circular Orbit

The position of a satellite in an equatorial circular orbit (i.e. the orbital plane is aligned with the Earth's equatorial plane) can be written as

$$\boldsymbol{r}^i(\theta) = r \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \\ 0 \end{bmatrix}$$

where the radius r and true anomaly θ are the polar coordinates of the satellite in its orbital plane. To simulate this orbit we need to propagate θ. In a circular orbit the radius is constant and equal to the semi-major axis

$$r(\theta) = r$$

The orbital period is then given by

$$T = \frac{2\pi}{\sqrt{\mu}} r^{\frac{3}{2}}$$

with $\mu = 398600\ \mathrm{km}^3/(\mathrm{kg}\cdot\mathrm{s}^2)$. Since the orbit is circular the mean anomaly is equal to the true anomaly such that

$$M_e = \theta = \frac{2\pi}{T} t \tag{1}$$

where t is the current time[1]. This can be converted to a differential equation $\dot{x} = f(t, x)$ by taking the derivative on both sides

$$\dot{\theta} = \frac{2\pi}{T}$$

Alternatively you can just propagate the time and update θ according to (1).

2.3 Simple Rotations

To facilitate visualization of change in orientations, it is common to use rotation matrices, where simple rotations are defined as

$$\mathbf{R}_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{bmatrix}$$
$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}$$
$$\mathbf{R}_z(\psi) = \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

where ϕ, θ, and ψ are the roll, pitch and yaw angles respectively. They can also be represented using quaternions

$$\boldsymbol{q}_1 = \begin{bmatrix} \cos\frac{\phi}{2} \\ \sin\frac{\phi}{2} \\ 0 \\ 0 \end{bmatrix}, \qquad \boldsymbol{q}_2 = \begin{bmatrix} \cos\frac{\theta}{2} \\ 0 \\ \sin\frac{\theta}{2} \\ 0 \end{bmatrix}, \qquad \boldsymbol{q}_3 = \begin{bmatrix} \cos\frac{\psi}{2} \\ 0 \\ 0 \\ \sin\frac{\psi}{2} \end{bmatrix}.$$

2.4 Simulating Earth's Rotation

Earth Centered Earth Fixed (ECEF) is denoted by superscript e and it moves with the rotation of the Earth around the $\mathbf{z}^i$ axis. The angular speed of the Earth is $\omega_{ie} = 7.2921 \cdot 10^{-5}$ rad/s, enabling the rotation matrix from ECEF to ECI to be written as

$$\mathbf{R}_e^i = \begin{bmatrix} \cos(\theta_G + \omega_{ie}t) & -\sin(\theta_G + \omega_{ie}t) & 0 \\ \sin(\theta_G + \omega_{ie}t) & \cos(\theta_G + \omega_{ie}t) & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

where θ_G is the initial angle between ECEF and ECI frame. Alternatively this can be expressed as

$$\dot{\boldsymbol{q}}_{ie} = \frac{1}{2}\boldsymbol{q}_{ie} \otimes \begin{bmatrix} 0 \\ 0 \\ \omega_{ie} \end{bmatrix}, \qquad \boldsymbol{q}_{ie}(t_0) = \begin{bmatrix} \cos\frac{\theta_G}{2} \\ 0 \\ 0 \\ \sin\frac{\theta_G}{2} \end{bmatrix}$$

[1] Generally it is the time since the last periapsis crossing, but in a circular orbit there is no periapsis. Additionally for a circular orbit we don't have to solve kepler's equation, so it is fine to just let t be the current time

2.5 Forward Euler's Method

A system of the form

$$\dot{x} = f(t, x)$$

can be solved numerically using the forward Euler's method defined as

$$x_{k+1} = x_k + hf(t, x_k), \qquad x_0 = x(0) \tag{2}$$

where x_k is the state at time t_k and $h = t_k - t_{k-1}$ is the step size. Using this we can compute the position of the satellite at any time t_k as long as we know the initial state $x(0)$. We will apply this to the previous section to create the animation.

3 Development Environment

In this course there are no restrictions on what kind of Integrated Development Environment the student uses to develop their simulator (only that python is used). The only prerequisite is that python3[2] is installed with the virtual environment[3] package (venv). In this section we will explain how to set up a minimal development environment that can run the assignment template that is available on canvas. We divide the process into two steps.

3.1 Step 1.

Make sure that python3 is installed on the computer. If it is not already installed, follow the install instructions at:

Windows: https://docs.python.org/3.14/using/windows.html

MacOS: https://docs.python.org/3.14/using/mac.html

Linux: Usually comes preinstalled

Python usually comes with a package manager called *pip* that allow users to install a package through the syntax

```
python3 -m pip install *package-name*
```

where a package is usually some library such as numpy. We will use pip to install the required packages in our virtual environment.

Note: It is not necessary to explicitly installed *pip* if you don't have it, since venv will automatically install it to any created virtual environments.

3.2 Step 2.

Download the template project from canvas and extract the contents into any desired folder, for instance:

```
assignment1/
├── 3DModels
│   ├── satellite.obj
│   ├── earth.obj
│   └── earth_texture.jpg
├── Data
│   └── .
├── requirements.txt
├── orbit_lib.py
├── plotter.py
├── simutils.py
└── sat_lib.py
```

[2]Latest stable release at the time of this document is 3.14.3, however having the latest version is not required.

[3]A virtual environment is an isolated local environment that allows Python users to install and upgrade Python packages within the virtual environment. This avoids interference with the behaviour of other Python applications running on the same system.

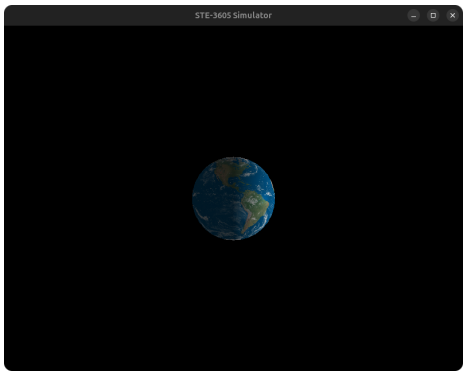

Figure 1: Assignment template.

```
├── simulator.py
└── assignment.py
```

Next open a terminal/powershell and navigate inside the folder to run the command

```
python3 -m venv venv
```

which will make a virtual environment resulting in the creation of an 'venv' folder that will contain any installed packages. Then activate the virtual environment in the current terminal by typing the command

```
source venv/bin/activate
```

With the virtual environment active in the current terminal, installing any packages using pip will only install them locally in the virtual environment. Installing the required packages can be done by running the command

```
pip install -r requirements.txt
```

which will look in the 'requirements.txt' file and install all the packages listed there. To test that everything is working you can run the command

```
python assignment.py
```

which should give the result shown in Figure 1.

Note: When working with the assignments there could be times where you need to know a bit about VisPy, therefore we have added a brief introduction to Vispy in Appendix B.

4 Assignment Template

In this section we will briefly explain the assignment template. Then we will outline the implementation of a equatorial orbit simulator, which will serve as an introduction for developing and implementing more complicated scenarios.

4.1 simulator.py

The general outline of simulator.py is the same as what you'll see in Appendix B, just with some added functionality. There are three main components: SimCanvas, Simulator and BaseScenario, which we will briefly explain in the next subsections.

4.1.1 SimCanvas class

The SimCanvas constructs a default scene which consists of the Earth, a satellite and three reference frames (ECI, ECEF and body). The Earth and satellite are custom meshes that are loaded from the '3DModel' folder. It also constructs a scene_list of all the visuals in the scene which is used when updating their poses (position+orientation) and scale. When the window initializes it also creates a timer which is used to periodically check a queue for data received by the simulator. If it has it will update the pose of the visuals in the scene. By default the camera is centered at the scene origin, but by pressing the 'h' key you can cycle through which visuals the camera should center on. There is also an 'anim_close' event that gets set when the window is closed. This event is used to signal to the simulator that the window has closed and to therefore stop the simulation.

4.1.2 Simulator class

The main responsibility of the simulator class is to propagate the simulation time and to publish data to the 'SimCanvas' instance. It accepts a configuration where the user can set the start time, end time and time step of the simulation. In addition it also includes 'anim_dt' which tells the simulator how often it should publish the current state to the 'SimCanvas' instance. Simulating satellite orbits in real-time is not always useful since they can have orbital periods of several hours, therefore we use a 'speed_factor' that can speed up the animation without having to increase the time step. When a 'Simulator' is initialized it creates a thread that will run the 'sim_runner' method which implements the simulation logic. The simulator relies on a scenario that implements the actual dynamics that is being simulated, this allows us to reuse the 'Simulator' class and not having to rewrite it whenever we want to simulation a new system.

Notice how the 'Simulator' does not rely on the existance of a 'SimCanvas' instance so that if you don't want visualization you simply do not create a 'SimCanvas' and don't call 'app.run()'. The simulation finishes when either when the 'anim_close' event is set or the simulation time reaches the end time.

4.1.3 BaseScenario class

The 'BaseScenario' describes the interface that a simulator uses to simulate a system. When creating your own scenarios you simply inherit from 'BaseScenario' and override the 'init', 'update', 'get' (required) and 'post_process' (optional) methods.

The 'init' method is called before the simulation starts and is used to set the initial conditions. The 'post_process' method gets called after the simulation is done and it is mainly intended to store simulation results for plotting. The 'update' method is the heart of the simulation system where the dynamics is evaluated and then numerically integrated to update the system state. The 'get' function returns a list with format (This is what is sent to 'SimCanvas'.)

```
[[name, position, orientation], ...]
```

where name corresponds to the scene_list name of the visual you want to update. For instance if you'd want to update the satellite and ECEF frame the list could look like

```
[['satellite',satellite_position,satellite_orientation], ['ecef frame',ecef_position,ecef_orientation]]
```

Therefore any visuals not mentioned in the list will not get updated. This implies that it is our responsibility to implement the rotation of the Earth and ECEF frame.

Note: This is a weakness of the current implementation that the hard coded names have to correspond between the scenario and 'SimCanvas'. Although this can be improved by implementing a better configuration system that uses a serialization library such as yaml or json to dynamically construct scenes. Another weakness is that 'SimCanvas' for now only accepts quaternions for orientation (alhtough you can manually change this). This could also be fixed by using a serialization format.

4.2 Plotter.py

Used to plot simulation results.

4.3 simutils.py

Here we put helper functions and classes that the rest of the project can depend on. For instance 'rotscaleloc_to_vispy' developed in Appendix B is defined here, as well as a quaternion class.

4.4 orbit_lib.py

Functions and classes related to orbital mechanics.

4.5 sat_lib.py

Functions and classes related to satellite attitude dynamics.

4.6 assignment.py

This is where you define the scenario for different assignments. For each assignment you will usually extend the functionality of the various python files from previous assignments. Then make a new assignment#.py file that implements the specific scnario for that assignment. It is therefore it is a good idea to use a version control tool such as git to track changes. Alternatively you could have a separate folder for each assignment and to avoid having a virtual environment for each assignment (each one would use about 400-500 MB of diskspace) you can create the virtual environment at the same folder level as the assignment folders then you just activate the virtual environment from there and navigate to the assignment folder you are currently working in.

Table 1: Orbit parameters for assignment 1.

name	value
r	$R_E + 400$
t_0	0
t_{step}	1
t_e	T
speed_factor	100
anim_dt	0.04
scale_factor	1000

4.7 Equatorial orbits

In this section we will implement an equitorial orbit scenario with parameters given in Table 1, where T refers to the orbital period. The initial conditions are set to $\theta(0) = 0$, $\theta_G = 0$. The basic structure of an assignment is shown in Listing 1.

```
import numpy as np
import simutils as su
import sat_lib as sl
import orbit_lib as ol
import simulator as sim
class ScenarionAssignment1(sim.BaseScenario):
  def init(self,t):
    #Initialization
  def update(self,t,dt):
    #Propagate states
  def get(self):
    #Return the current states:
    #[['name1',p1,q1],['name2',p2,q1],...]
    return []
  def post_process(self,t,dt):
    #Write log files
def main():
  scenario_config = {'t_0':0,'t_e':1,'t_step':1,'
    speed_factor':1,'anim_dt':1/25,'scale_factor':1,'
    visualise':True}
  scenario = ScenarioAssignment1()
  sim.create_and_start_simulation(scenario_config,scenario)
if __name__ == "__main__":
  main()
```

Listing 1: Basic assignment template.

We know from the previous sections that

$$\boldsymbol{r}^i(\theta) = r \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \\ 0 \end{bmatrix}$$
$$T = \frac{2\pi}{\sqrt{\mu}} r^{\frac{3}{2}}$$
$$\dot{\theta} = \frac{2\pi}{T}$$
$$\dot{\boldsymbol{q}}_{ie} = \frac{1}{2}\boldsymbol{q}_{ie} \otimes \begin{bmatrix} 0 \\ 0 \\ \omega_{ie} \end{bmatrix}$$

combining this with the forward Euler's method to update $\dot{\theta}$ and $\dot{\boldsymbol{q}}_{ie}$ gives us the equations

$$\theta_{k+1} = \theta_k + t_{step} \cdot \dot{\theta}_k$$
$$\bar{\boldsymbol{q}}_{ie,k+1} = \boldsymbol{q}_{ie,k} + t_{step} \cdot \dot{\boldsymbol{q}}_{ie,k} \rightarrow \boldsymbol{q}_{i,k+1} = \frac{1}{\|\bar{\boldsymbol{q}}_{i,k+1}\|}\bar{\boldsymbol{q}}_{i,k+1}$$

with

$$\dot{\theta}_k = \frac{2\pi}{T}$$
$$\dot{\boldsymbol{q}}_{ie,k} = \frac{1}{2}\boldsymbol{q}_{ie,k} \otimes \boldsymbol{\omega}^i_{ie,k}, \quad \boldsymbol{\omega}^i_{ie,k} = \begin{bmatrix} 0 \\ 0 \\ \omega_{ie} \end{bmatrix}$$

and from this we get the position of the satellite (and the body frame) at t_k

$$\boldsymbol{r}^i_k = r \begin{bmatrix} \cos(\theta_k) \\ \sin(\theta_k) \\ 0 \end{bmatrix}.$$

We can now construct the return value for the 'get' method

```
[['satellite',r_i,self.q],['body frame',r_i,self.q],['earth'
    ,np.zeros(3),self.q_E],['ECEF frame',np.zeros(3),self.q_E
    ],['ECI frame',np.zeros(3),su.Quaternion()]]
```

if we want to plot some results we can store values for $\boldsymbol{r}^i$ in a numpy array and then in 'post_process' write it to a file using 'su.log_pos'.

```
#Initializing the numpy array
self.pos_plot = np.array([t,self.r*np.cos(self.theta),self.r
    *np.sin(self.theta),0])
#Updating it with new values
self.pos_plot = np.vstack((self.pos_plot,np.array([t,self.r*
    np.cos(self.theta),self.r*np.sin(self.theta),0])))
#Writing results
su.log_pos('assignment1_position',self.pos_plot)
```

this will store a log file in 'data/' folder. Then we can use plotter.py with the command

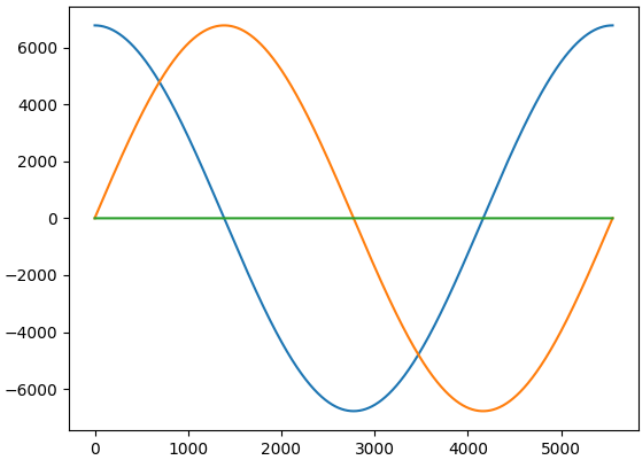

Figure 2: Plot of satellite position.

```
python plotter.py lineplot data/*name_of_log_file*
```

which should give the result show in Figure 2.

Assignment 1

1. Implement the equatorial orbit scenario from this assignment.
2. Show me the result, either in person or by making a video of the result and uploading it to canvas.

A Python

Python applies the following syntax for programming.

A.1 If-Sentence

An if-sentence can be handled in Python as shown below which gives an output value that is limted between an upper and lower bound.

```
# Example if sentence
value = 10
min_value = -5
max_value = 5

if value >= max_value:
  output = max_value
elif value <= min_value:
  output = min_value
else:
  output = value
```

A.2 For-Loop

A for loop can be created for a range of values as

```
# Using a for loop with range
for number in range(1,6):
  print("This is number ", number)
```

Note that the range(1,6) gives a range between 1 and 5, as the start is inclusive, but stop is exclusive. It is also possible to loop through strings, e.g.

```
# Using a for loop with a string
fruits = ["Apple", "Pear", "Banana"]
for fruit in fruits:
  print("The fruit is ", fruit)
```

A.3 While Loop

Similarly as a for loop, a while loop can be applied as

```
# Using a while loop
value = 10
lower_bound = 0.1

while value > lower_bound:
  print("The value is ", value)
  value = value - 0.05

print("The final value is ", value)
```

A.4 Functions

A Python function can be created as follows:

```
# Creating a function
def my_python_function(argument1, argument2):
  output1 = argument1 + argument2
  output2 = argument1 - argument2

  return output1, output2
```

Then the function can be called as

```
# Calling the function
arg1 = 10
arg2 = 2
out1, out2 = my_python_function(arg1, arg2)
```

allowing access to the variables out1 and out2. In terms of Python, the key item to keep control of is the tabulation as all lines within a function must be tabulated one step. This allows functions within functions as long as the tabulation is maintained.

A.5 Numpy

A matrix can be constructed using the Numpy library as

```
# Creating a matrix
import numpy as np
matrix = np.array([[1,2,3],
          [4,5,6],
          [7,8,9]])
```

and a vector can be created as

```
# Creating the vector
vector = np.array([1,2,3])
```

and they can be multiplied together using either @ or $.dot()$ as

```
# Matrix-Vector product
new_vector = matrix @ vector
new_vector = matrix.dot(vector)
```

A.6 Recommended Notation

When dealing with rotation matrices in software, it is easy to lose track of what is rotating from and to. It is therefore recommended to apply the following notation for the following formula:

$$\mathbf{v}^{to} = \mathbf{R}^{to}_{from}\mathbf{v}^{from} \tag{3}$$

as

```
# Example showing how to perform rotations in Python
import numpy as np
v_to = R_to_from @ v_from
```

such that a rotation from ECEF to ECI frame, $\mathbf{v}^i = \mathbf{R}_e^i \mathbf{v}^e$, results in

```
# Example showing how to calculate a vector in ECI frame
import numpy as np
v_i = R_i_e @ v_e
```

This allows rotation matrices and (later) quaternions to apply the same notation which makes things much easier to keep track of. Note that here numpy is used to allow matrix/vector multiplications.

B Introduction to Vispy

In this section we will give a very brief introduction to VisPy which is used as the visualisation backend of the simulator. The reason we are using VisPy is that out of the python packages with 3D visualization functionality that we've tried:

- Pyvista
- Open3D
- Matplotlib
- PyVista
- Mayavi

it has shown to have the best average score in terms of being light-weight, performant and easy to use. It is also well integrated with pyqtgraph which makes it possible to extend the simulator with a user interface and real-time plotting. The source code and documentation for VisPy is found at

Source: `https://github.com/vispy/vispy`

Documentation: `https://vispy.org/index.html`

Although there exist some interesting alternatives like pygfx, glumpy and datoviz they seem at the moment less mature and not as easy to set up and use.

In general there are two ways to use VisPy

Low-Level: VisPy exposes an OpenGL type API to the user called gloo which allows for custom and performant visualizations. This is a very involved approach and requires knowledge of OpenGL/GLSL.

High-Level: VisPy offers a high-level API that allows users to create a wide variety of visualizations without needing to know OpenGL/GLSL. Easier to work with, but less flexible.

In this document we will only cover the high-level approach, which is sufficient for our needs.

At the core of VisPy there is an 'Application' which handles event logic (creating and closing the window, reacting to keyboard and mouse inputs etc) and a 'Canvas' that displays 'Widgets' or GUI elements. Out of all the possible widgets the most relevant one for us is the 'ViewBox' where we can insert different types of 'Visuals' (for instance cubes, cpheres, custom geometry etc.) for display.

To create a custom visualisation we extend the 'SceneCanvas' class (which extends the 'Canvas' class). The 'SceneCanvas' class includes the necessary functionality to create a scene, which is essentially a list of visuals to display, a coordinate system and a renderer. The essential steps are to add a 'ViewBox' widget to the SceneCanvas and insert visuals into the scene by adding them to the 'ViewBox' as shown in Listing 4.

```
1  from vispy import app, scene
2  class MyCanvas(scene.SceneCanvas):
3    def __init__(self):
4      scene.SceneCanvas.__init__(self,title='XYZAxis Test',
       keys='interactive',size=(800,600))
5      self.unfreeze()
6      self.view = self.central_widget.add_view(camera='arcball
       ')
7      self.frame = scene.visuals.XYZAxis()
8      self.view.add(self.frame)
9      self.freeze()
10     self.show()
11
12 canvas = MyCanvas()
13 app.run()
```

Listing 2: Visualizing a reference frame

This simple example includes most of what you need to know to use VisPy so we spend some additional time explaning the code in detail

Line 4: We call the 'SceneCanvas' constructor, which sets up everything in the backend needed to eventually create and display a window with a given title and size (keys argument is described later). It also sets 'MyCanvas' instance as the 'native_canvas' (see Line 13).

Line 5, Line 9: We have to unfreeze the canvas to be able to add additional properties to it. Then when we are done modifying it, we freeze it.

Line 6: Adds a ViewBox widget that allows us to see the scene within the canvas. Making it the 'central_widget' means it will fill the whole canvas. The scene is viewed using a camera and in this case we add an 'arcball' camera which on user input will rotate around a specified point while looking at it (by default this point is the scene origin).

Line 7: Creates an XYZAxis 'Visual'. Like most 'Visuals' it can be rotated, translated and scaled using transforms. It can also be parented to other visuals such that it moves with the parent.

Line 8: Adds the 'Visual' to the scene which uses the transform property of the 'Visual' and its parent relation to determine its location, orientation and scale within the scene.

Line 10: By default a 'Canvas' is hidden, calling 'show()' makes it visible in the window it is attached to.

Line 13: This creates the window and starts the backend event loop. When creating the window it checks if a 'native_canvas' exists and attaches it to the window.

Try to experiment with this code for instance by trying other types of cameras or adding other visuals. Some possibilities

Camera types: 'fly','panzoom','turntable','arcball'

Visuals: Arrow, Axis, Cube, Ellipse, Sphere, XYZAxis, Mesh, Tube

The backend event loop allows the application to react to keyboard and mouse inputs, timers, window resizing, closing and more. It is possible to define functions that should run when a specific key is pressed, this is what 'keys='interactive" does. It defines that the application should close when the 'Esc' key is pressed and to go fullscreen when 'F11' is pressed. It is possible to define a dictionairy that maps keys to specific functions, for instance by modifying the previous example

```
from vispy import app, scene
    ⋮
  self.key_map = {'f': self.f_function}
  scene.SceneCanvas.__init__(self,title='XYZAxis Test',keys=
    self.key_map,size=(800,600))
    ⋮
def f_function(self):
  print('f key pressed')
    ⋮
```

Listing 3: Custom keys

However, by doing this we can no longer close the window by pressing 'Esc'. A better approach is to use the 'on_key_press' function which is automatically called when a keypress event occurs

```
from vispy import app, scene
class MyCanvas(scene.SceneCanvas):
  def __init__(self):
    scene.SceneCanvas.__init__(self,title='XYZAxis Test',
   keys='interactive',size=(800,600))
    self.unfreeze()
    self.view = self.central_widget.add_view(camera='arcball
   ')
    self.frame = scene.visuals.XYZAxis()
    self.view.add(self.frame)
    self.freeze()
    self.show()

  def on_key_press(self,event):
    if event.text == 'f':
      print('f key pressed')

canvas = MyCanvas()
app.run()
```

Listing 4: Key events

Now the application reacts to the 'f' key as well as 'Esc' and 'F11'!

B.1 Coordinate Transformations with VisPy

We want to be able to move visuals around and to do this we need to apply transformations to them. VisPy uses homogenuous tranformation matrices to represent rigid motions through the transforms module. Let us extend the previous example to allow us to rotate the reference frame by pressing the 'a' and 'd' keys

```
import numpy as np
from vispy import app, scene
from vispy.scene import MatrixTransform as Mat4
from vispy.util.quaternion import Quaternion as Quat
class MyCanvas(scene.SceneCanvas):
    def __init__(self):
        scene.SceneCanvas.__init__(self,title='Simulator',keys='interactive',size=(800,600))
        self.unfreeze()
        self.view = self.central_widget.add_view(camera='arcball')
        self.frame = scene.visuals.XYZAxis()
        self.view.add(self.frame)
        self.theta = 0
        self.delta = np.pi/10
        self.freeze()
        self.show()

    def on_key_press(self,event):
        if event.text == 'd':
            self.theta += self.delta
        elif event.text == 'a':
            self.theta -= self.delta
        ctheta = np.cos(self.theta/2)
        stheta = np.sin(self.theta/2)
        q = Quat(w=ctheta,x=0,y=0,z=stheta)
        self.frame.transform = Mat4(q.get_matrix())

canvas = MyCanvas()
app.run()
```

Listing 5: Vispy Coordinate Transformation Example

Here we initialize 'theta' to 0 and we increment it by 'delta' when we press the 'd' key and decrement it by 'delta' when we press the 'a' key. We would expect from this that when we press the 'd' key the 'frame' should rotate counter-clockwise around the z-axis since 'theta' is incremented by 'delta'. However, as you are observing it is actually rotating clockwise. The reason for this is that VisPy stores the transformation matrices in column major form as opposed to the more commonly used row major form. In other words when supplying a transformation to VisPy we need to transpose it first. In the above example this would correspond to conjugating the quaternion

```
        q = Quat(w=ctheta,x=0,y=0,z=-stheta)
```

To further illustrate this we provide two more examples in Listing 6 and Listing 7 where the resulting transformation 'H' end up being the same for both. Notice

how without using the VisPy transform module we need to supply the transpose of the homogenuous transformation matrix.

```
H = Mat4()
H.rotate(90,(0,0,1))
H.translate((2,0,0))
```

Listing 6: Using Vispy transforms

```
psi = np.pi/2
cpsi = np.cos(psi)
spsi = np.sin(psi)
R = np.array([[cpsi,-spsi,0],[spsi,cpsi,0],[0,0,1]])
p = np.array([[2,0,0]]).T
#combine R and p into [R p;0 1]
HT = np.vstack(((np.hstack((Rot,p)),np.array
([[0,0,0,1]]))))
H = Mat4(matrix=HT.T)
```

Listing 7: Without using Vispy transforms

The reason for bringing this up is that typically we want to keep the visualization and simulator decoupled. Making sure the simulator does not depend on the visualization framework makes makes it easier to switch it out if necessary or to even run the simulation without any visualization. In addition within the satellite simulator we will not be using homogenuous transformation matrices and therefore we need a function that can take a position and orientation from the simulator as input and output a proper VisPy homogenuous transformation matrix. Based on the previous discussion the function could look like

```
def rotscaleloc_to_vispy(pos=None,quat=None,Rot=None,Eul=
    None,scale=None):
  if quat is not None:
    q = Quat(w=quat[0],x=quat[1],y=quat[2],z=quat[3])
    H = Mat4(q.conjugate().get_matrix())
  elif Rot is not None:
    p = np.array([[0,0,0]]).T
    HT = np.vstack(((np.hstack((Rot,p)),np.array
    ([[0,0,0,1]]))))
    H = Mat4(HT.T)
  elif Eul is not None:
    q = Quat.create_from_euler_angles(Eul[2],Eul[1],Eul[0])
    H = Mat4(q.conjugate().get_matrix())
  else:
    H = Mat4()
  if scale is not None:
    H.scale((scale,scale,scale))
  if pos is not None:
    H.translate(pos)
  return H
```

Listing 8: Position and Orientation to Vispy transform

where we have also included support for several attitude representations and scaling since it is useful to have sometimes.

[3]Keep in mind that the transpose of a homogenuous transformation matrix is not the same as its inverse like it is for rotation matrices

B.2 Animating

Up to this point we have learned how to add visuals to our scene and given them a position and orientation, but we have yet to show motion. We can accomplish this by using timers that generate timeout events

```
MyTimer = app.timer.Timer()
```

The 'Timer' constructor supports the following arguments

interval default='auto', time between timeout events

connect default=None, function to call when a timeout event occurs

iteration default=-1, amount of timeout events before the timer will stop

start default='false', set to 'true' to immediatly start the timer

app default='None', the application to attach the timer to

Combining this with the knowledge from the previous sections, we can construct a simple example shown in Listing 9 where the frame rotates around the z-axis. In addition pressing the 's' key will toggle between clockwise and counter-clockwise rotation.

```
import numpy as np
import simutils as su
from vispy import app, scene
from vispy.scene import MatrixTransform as Mat4
from vispy.util.quaternion import Quaternion as Quat
class MyCanvas(scene.SceneCanvas):
  def __init__(self):
    scene.SceneCanvas.__init__(self,title='XYZAxis Test',keys='interactive',size=(800,600))
    self.unfreeze()
    self.view = self.central_widget.add_view(camera='arcball')
    self.frame = scene.visuals.XYZAxis()
    self.view.add(self.frame)
    self.theta = 0
    self.w = 1
    self._timer = app.timer.Timer(interval = 1/30,connect=self.on_timer,start=True,app=self.app)
    self.freeze()
    self.show()

  def on_timer(self,event):
    self.theta += self.w*event.dt
    q = Quat.create_from_euler_angles(rx=self.theta,ry=0,rz=0)
    self.frame.transform = Mat4(q.conjugate().get_matrix())
    self.update()

  def on_key_press(self,event):
      if event.text == 's':
          self.w *=-1

```

```
canvas = MyCanvas()
app.run()
```

Listing 9: Simple animation example using timers

B.3 Thread and Queues

As we mentioned previously we want our simulation to be decoupled from the visualization. We can do this by running the simulator in its own separate thread and use a queue to pass data from the simulator to VisPy. Listing 10 shows the modules that needs to be imported to use threads. The decoupled version 'MyCanvas' is shown in Listing 11

```
import numpy as np
import simutils as su
from vispy import app, scene
from vispy.scene import MatrixTransform as Mat4
from vispy.util.quaternion import Quaternion as Quat
import threading
import queue
import time
```

Listing 10: Position and Orientation to VisPy transform

```
class MyCanvas(scene.SceneCanvas):
  def __init__(self,anim_queue,anim_dt,anim_close):
    scene.SceneCanvas.__init__(self,title='XYZAxis Test',keys='interactive',size=(800,600))
    self.unfreeze()
    self.anim_queue = anim_queue
    self.anim_dt = anim_dt
    self.anim_close = anim_close
    self.view = self.central_widget.add_view(camera='arcball')
    self.frame = scene.visuals.XYZAxis()
    self.view.add(self.frame)
    self._timer = app.timer.Timer(interval = anim_dt,connect=self.on_timer,start=True,app=self.app)
    self.freeze()
    self.show()


  def on_timer(self,event):
    if not self.anim_queue.empty():
      p,q = self.anim_queue.get_nowait()
      self.frame.transform = rotscaleloc_to_vispy(pos=p,quat=q)
      self.update()

  def on_close(self,event):
    self.anim_close.set()
    return super().on_close(event)
```

Listing 11: Position and Orientation to Vispy transform

The simulation thread is just a function as shown in Listing 12

```
def sim_runner(anim_queue,anim_dt,sim_close):
  t_step = 1/100
  t_e = 10
  t = 0
  t_anim = 0
  theta = 0
  w = 1
  while not sim_close.is_set() and t < t_e:
    theta += w*t_step
    if t-t_anim > anim_dt:
      t_anim = t
      p = np.array([0,0,0])
      q = su.Quaternion(self.theta,[0,0,1])
      anim_queue.put((p,q))
      time.sleep(0.01)
    t += t_step
```

Listing 12: Position and Orientation to Vispy transform

Listing 13 shows how to set up a simulation thread and a VisPy app that communicate through a queue. The 'anim_close' is used by the VisPy app to notify the simulation if the window is closed. Otherwise the program will keep running until the simulator finishes the simulation.

```
anim_dt = 1/30
anim_close = threading.Event()
anim_queue = queue.SimpleQueue()
sim_thread = threading.Thread(target=sim_runner,args=[
    anim_queue,anim_dt,anim_close])
canvas = MyCanvas(anim_queue,anim_dt,anim_close)
sim_thread.start()
app.run()
```

Listing 13: Position and Orientation to Vispy transform